

Labo 6 – Implémentation d'une Saga Orchestrée et Gestion de la Machine d'État

Cours : **Architecture Logicielle (LOG430)**

Session : **Été 2025**

Date du laboratoire : **Semaine du 23 Juin 2025**



Le génie pour l'industrie

Contexte et objectifs du laboratoire

Ce laboratoire prolonge les travaux des Labos 4 et 5. Après avoir introduit l'API Gateway, la répartition de charge, l'observabilité et la séparation logique en microservices, vous allez maintenant implémenter une **Saga orchestrée synchrone** pour coordonner une **transaction répartie entre plusieurs microservices**.

L'objectif pédagogique principal est de vous familiariser avec les mécanismes de gestion d'une suite d'actions dépendantes, notamment via une orchestration centrale synchrone, et de bien contrôler les événements, les états intermédiaires et la résilience en cas d'échec d'un ou plusieurs sous-processus.

Objectifs pédagogiques

- Comprendre le concept de Saga orchestrée **synchrone** pour la gestion des transactions distribuées.
- Implémenter une orchestration centralisée et synchrone entre 3 à 4 microservices.
- Mettre en place un suivi de la machine d'état d'une entité métier (par exemple : Commande).
- Gérer les événements métiers, les échecs partiels et les compensations.

Travail demandé

1. Définition du scénario métier

Choisissez un scénario métier concret impliquant plusieurs microservices. Exemple recommandé :

- Création de commande client :
 1. Vérification du stock
 2. Réservation de stock
 3. Paiement
 4. Confirmation ou annulation de commande

2. Mise en place de la Saga orchestrée

- Créez un service **orchestrateur synchrone** dédié à la gestion de la saga.
- Ce service reçoit un événement de début de transaction (ex : **CommandeCréée**).
- Il déclenche les appels aux microservices concernés **de manière synchrone**, en suivant un enchaînement contrôlé d'étapes.
- En cas de succès, la saga termine avec succès et l'état final est **CommandeConfirmée**.
- En cas d'échec (ex : paiement refusé), exécutez les actions de compensation (ex : libérer le stock).

3. Gestion des événements et de l'état

- Chaque microservice doit publier un événement de succès ou d'échec après traitement.
- Le service orchestrateur doit conserver l'état courant de la saga et mettre à jour la machine d'état de la commande.
- Implémentez une machine d'état explicite (ex : Enum + logs + persistée si possible).

4. Simulation de cas d'échec

- Introduisez des échecs contrôlés (ex : stock insuffisant, paiement en échec).
- Observez les effets sur la saga et la machine d'état.
- Implémentez des actions de rollback ou de compensation.

5. Observabilité et traçabilité

- Ajoutez des métriques Prometheus pour le suivi des sagas (durée moyenne, échecs, étapes atteintes).
- Visualisez l'évolution des états via Grafana.
- Tracez les événements métiers et décisions dans les logs structurés.

Livrables attendus

- Code source complet dans le dossier lab6/
- Un README clair avec instructions de déploiement, tests et cas d'échec simulés.
- Un rapport structuré décrivant :
 - Le scénario métier et la saga implémentée
 - Le diagramme de la machine d'état
 - Les décisions d'architecture (2 ADR minimum)
 - Les mécanismes de compensation
 - Les captures d'écran des dashboards

Conseils

- Gardez la saga simple (4 étapes maximum).
- Utilisez une API synchrone pour les appels entre services ; la logique de contrôle doit rester centralisée dans l'orchestrateur.
- Privilégiez des appels directs avec gestion explicite des erreurs à chaque étape (pas de file de messages dans ce labo).
- Utilisez des événements clairs : `StockRéservé`, `PaiementRéussi`, `PaiementÉchoué`, etc.
- Testez les transitions d'état avec attention : pas de commande "bloquée" !